

Foundational Invites System - Standard Operating Procedure

Document Version: 1.0
Last Updated: February 1, 2025
Owner: Operations / Finance
Classification: Internal Use Only

Table of Contents

- 1. [System Overview](#)
 - 2. [Architecture](#)
 - 3. [Frontend Components](#)
 - 4. [Backend API](#)
 - 5. [Database Schema](#)
 - 6. [User Workflows](#)
 - 7. [Administrative Procedures](#)
 - 8. [Security & Compliance](#)
 - 9. [Troubleshooting](#)
 - 10. [Maintenance](#)
-

System Overview

Purpose

The Foundational Invites System enables Crave'n Inc to accept friends & family support contributions through an invite-only payment flow. The system maintains strict compliance by keeping equity language off public pages and enforcing \$50-\$500 contribution limits.

Key Features

- **Invite-Only Access:** All payment flows require a valid access code
- **Amount Constraints:** \$50 minimum, \$500 maximum (enforced at database level)
- **Admin Management:** Hub portal interface for invite creation and management
- **Stripe Integration:** Secure payment processing via Stripe Checkout
- **Status Tracking:** Full lifecycle tracking (invited → accepted → paid)

Business Rules

Rule	Value	Enforcement Level
Minimum Amount	\$50 (5000 cents)	Database trigger + API validation
Maximum Amount	\$500 (50000 cents)	Database trigger + API validation
Access Method	Invite code only	Server-side verification

Rule	Value	Enforcement Level
Public Equity Language	Prohibited	Content review required
Admin Access	CEO/Admin only	BusinessAuthGuard + permission checks

Architecture

Technology Stack

Frontend:

- React 18.3.1
- React Router 6.26.2
- Vite 7.1.12
- TailwindCSS 3.4.11

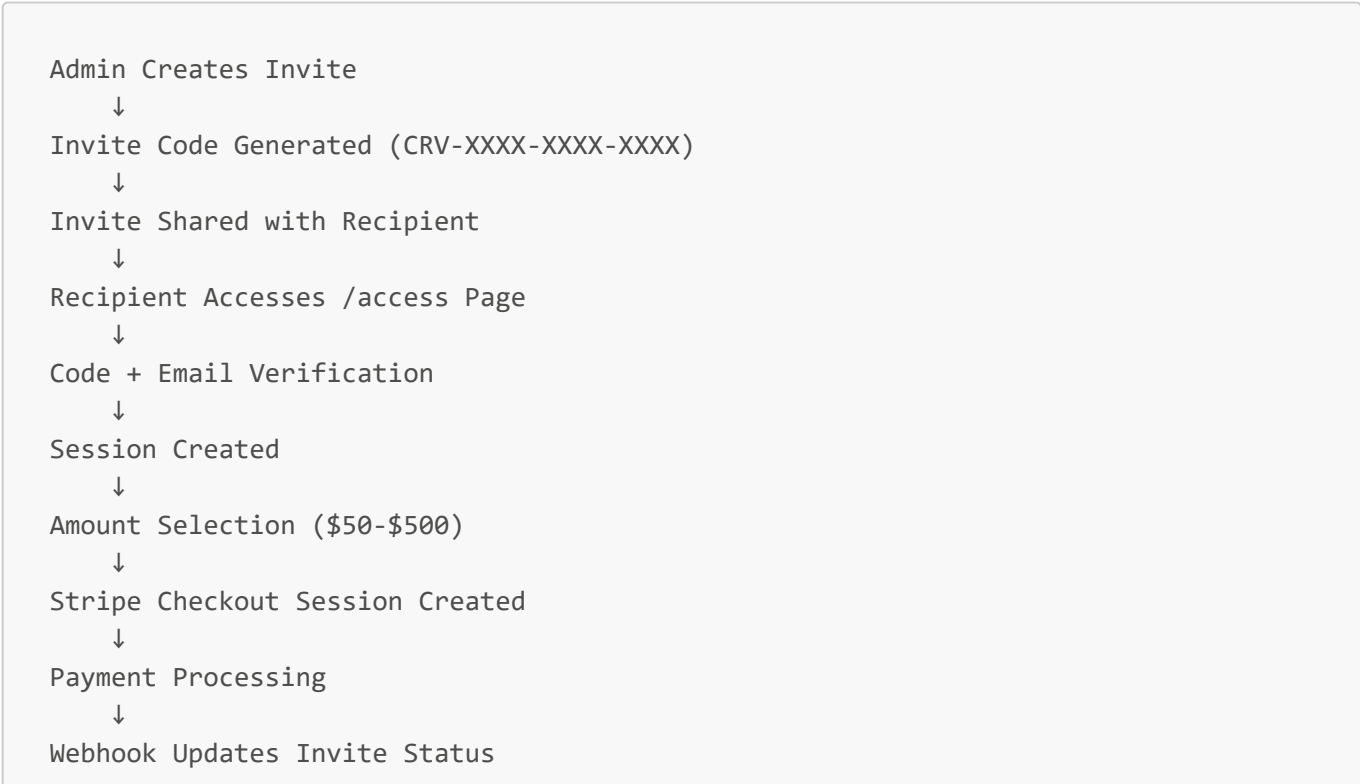
Backend:

- Express 4.19.2
- TypeScript 5.5.3
- Supabase Client (service role)
- Stripe SDK 17.4.0

Database:

- PostgreSQL (via Supabase)
- Row Level Security (RLS) enabled
- Database triggers for amount validation

System Flow





Confirmation Page

Frontend Components

Public Pages

/support - Landing Page

File: `src/pages/Support.tsx`

Purpose: Public-facing entry point with no equity language.

Features:

- "Private access by invitation only" messaging
- CTA button to `/access` page
- No financial or equity terminology

Access: Public (no authentication required)

/access - Access Code Verification

File: `src/pages/Access.tsx`

Purpose: Verify invite code and email, create authenticated session.

User Input:

- Access code (CRV-XXXX-XXXX-XXXX format)
- Email address
- Relationship note (optional, for internal tracking)

API Call:

- `POST /api/support/verify-access`
- Payload: `{ accessCode, email }`

Session Storage:

- Stores invite session data for `/allocate` page
- Key: `invite_session`
- Data: `{ inviteId, email, minAmount, maxAmount }`

Validation:

- Access code must match invite record
- Email must match invite record
- Invite must not be revoked or expired
- Invite must not already be paid

Error Handling:

- Invalid code/email: "Invalid access code or email"
 - Revoked: "This invite has been revoked"
 - Expired: "This invite has expired"
 - Already paid: "This invite has already been used"
-

/allocate - Amount Selection

File: `src/pages/Allocate.tsx`

Purpose: Select contribution amount and proceed to payment.

Features:

- Preset amounts: \$50, \$100, \$250, \$500
- Custom amount input (validated against min/max)
- Terms acceptance checkbox
- Real-time amount validation

Session Requirements:

- Must have valid `invite_session` in sessionStorage
- Redirects to `/access` if session missing

API Call:

- `POST /api/support/create-checkout`
- Payload: `{ inviteId, amountCents, email }`
- Response: `{ checkoutUrl, sessionId }`

Validation:

- Amount must be between min and max (from invite record)
- Terms acceptance required
- Stripe Checkout redirect on success

Error Handling:

- Session expired: Redirect to `/access`
 - Invalid amount: Display error message
 - API failure: Display error, allow retry
-

/success - Payment Confirmation

File: `src/pages/Success.tsx`

Purpose: Display payment confirmation after successful Stripe checkout.

Features:

- Success message and icon
- Session ID display (for reference)
- Return to home button
- Clears sessionStorage on load

URL Parameters:

- `session_id`: Stripe checkout session ID
 - `invite_id`: Invite record ID
-

Admin Portal**`/hub/foundational/invites` - Admin Management**

File: `src/pages/HubFoundationalInvites.tsx`

Purpose: Create, view, and manage foundational invites.

Access Control:

- Protected by `BusinessAuthGuard`
- Permission check: Admin or CEO only
- Torrance has universal access

Features:**Create Invite Form:**

- Email (required)
- Full Name (optional)
- Relationship Note (optional, internal use)
- Expires At (optional date)

Invite List Table:

- Access Code (monospace font)
- Email
- Status badge (invited/accepted/paid/revoked)
- Paid amount (if applicable)
- Created timestamp
- Revoke button (if not already revoked)

API Endpoints Used:

- `GET /api/hub/invites/list` - Load all invites
- `POST /api/hub/invites/create` - Create new invite
- `POST /api/hub/invites/revoke` - Revoke existing invite

Status Workflow:

```
invited → accepted → paid
      ↓
    revoked (can occur at any stage)
```

Backend API

Server Configuration

Base URL: `http://localhost:3001` (development)

Production: `https://cravenusa.com/api`

Entry Point: `server/index.ts`

Route Registration:

```
app.use("/api/hub/invites", invitesRoute);
app.use("/api/support", supportRoute);
```

Invite Management API

Base Path: `/api/hub/invites`

Create Invite

Endpoint: `POST /api/hub/invites/create`

Authentication: Admin/CEO only (via `assertHubAdmin`)

Request Body:

```
{
  "email": "recipient@example.com",
  "fullName": "John Doe",
  "relationshipNote": "Childhood friend",
  "expiresAt": "2025-12-31"
}
```

Response:

```
{
  "invite": {
    "id": "uuid",
    "access_code": "CRV-XXXX-XXXX-XXXX",
    "email": "recipient@example.com",
  }
}
```

```
    "status": "invited",
    "created_at": "2025-02-01T12:00:00Z"
  }
}
```

Error Responses:

- 400: Invalid request (missing email, invalid format)
- 401: Unauthorized (not admin)
- 500: Database error

Implementation: `server/routes/invites.ts` (lines 21-64)

List Invites

Endpoint: `GET /api/hub/invites/list`

Authentication: Admin/CEO only

Query Parameters: None

Response:

```
{
  "invites": [
    {
      "id": "uuid",
      "access_code": "CRV-XXXX-XXXX-XXXX",
      "email": "recipient@example.com",
      "full_name": "John Doe",
      "status": "paid",
      "accepted_at": "2025-02-01T12:00:00Z",
      "paid_at": "2025-02-01T12:05:00Z",
      "paid_amount_cents": 10000,
      "created_at": "2025-02-01T11:00:00Z",
      "expires_at": null
    }
  ]
}
```

Limits:

- Returns up to 200 most recent invites
- Ordered by `created_at` descending

Implementation: `server/routes/invites.ts` (lines 67-88)

Revoke Invite

Endpoint: `POST /api/hub/invites/revoke`

Authentication: Admin/CEO only

Request Body:

```
{
  "id": "invite-uuid"
}
```

Response:

```
{
  "ok": true
}
```

Behavior:

- Updates invite status to "revoked"
- Revoked invites cannot be used for payment
- Historical record preserved

Implementation: `server/routes/invites.ts` (lines 91-118)

Support Flow API

Base Path: `/api/support`

Verify Access

Endpoint: `POST /api/support/verify-access`

Authentication: None (public endpoint, but validates invite)

Request Body:

```
{
  "accessCode": "CRV-XXXX-XXXX-XXXX",
  "email": "recipient@example.com"
}
```

Response:

```
{
  "invite": {
```

```
    "id": "uuid",
    "min_amount_cents": 5000,
    "max_amount_cents": 50000,
    "email": "recipient@example.com",
    "full_name": "John Doe"
  }
}
```

Validation Checks:

1. Access code exists and matches
2. Email matches invite record
3. Status is not "revoked"
4. Not expired (if `expires_at` set)
5. Status is not "paid"

Side Effects:

- If status is "invited", updates to "accepted"
- Sets `accepted_at` timestamp

Error Responses:

- `404`: Invalid access code or email
- `403`: Invite revoked, expired, or already paid

Implementation: `server/routes/support.ts` (lines 13-70)

Create Checkout Session

Endpoint: `POST /api/support/create-checkout`

Authentication: None (validated via invite session)

Request Body:

```
{
  "inviteId": "uuid",
  "amountCents": 10000,
  "email": "recipient@example.com"
}
```

Response:

```
{
  "checkoutUrl": "https://checkout.stripe.com/c/pay/...",
  "sessionId": "cs_test_..."
}
```

Validation:

1. Invite exists and is valid
2. Amount is within min/max bounds
3. Invite status allows payment

Stripe Configuration:

- Payment method: Card only
- Mode: One-time payment
- Success URL: `{APP_URL}/success?session_id={CHECKOUT_SESSION_ID}&invite_id={inviteId}`
- Cancel URL: `{APP_URL}/allocate?invite_id={inviteId}`
- Metadata: `{ invite_id, type: "foundational_support" }`

Error Responses:

- **400**: Invalid amount or request
- **403**: Invite no longer valid
- **404**: Invite not found
- **500**: Stripe API error

Implementation: `server/routes/support.ts` (lines 72-140)

Stripe Webhook Handler

Endpoint: **POST** `/api/support/webhook`

Authentication: Stripe signature verification

Purpose: Update invite status when payment completes

Event Handled: `checkout.session.completed`

Process:

1. Verify webhook signature
2. Extract `invite_id` from session metadata
3. Update invite record:
 - Status: "paid"
 - `paid_at`: Current timestamp
 - `paid_amount_cents`: Session amount

Security:

- Raw body parsing required (configured in `server/index.ts`)
- Signature verification via `STRIPE_WEBHOOK_SECRET`
- Rejects invalid signatures with 400 error

Implementation: `server/routes/support.ts` (lines 142-182)

Database Schema

Table: `public.invites`

Migration File: `supabase/migrations/20260201000001_create_foundational_invites.sql`

Schema:

Column	Type	Constraints	Description
<code>id</code>	UUID	PRIMARY KEY, DEFAULT <code>gen_random_uuid()</code>	Unique identifier
<code>access_code</code>	TEXT	UNIQUE, NOT NULL	CRV-XXXX-XXXX-XXXX format
<code>email</code>	TEXT	NOT NULL	Recipient email (lowercase)
<code>full_name</code>	TEXT	NULL	Optional recipient name
<code>relationship_note</code>	TEXT	NULL	Internal relationship tracking
<code>status</code>	TEXT	NOT NULL, DEFAULT 'invited', CHECK	invited/accepted/paid/revoked
<code>min_amount_cents</code>	INTEGER	NOT NULL, DEFAULT 5000	Minimum contribution (50.00)
<code>max_amount_cents</code>	INTEGER	NOT NULL, DEFAULT 50000	Maximum contribution (500.00)
<code>accepted_at</code>	TIMESTAMPTZ	NULL	When access code was verified
<code>paid_at</code>	TIMESTAMPTZ	NULL	When payment completed
<code>paid_amount_cents</code>	INTEGER	NULL	Actual payment amount
<code>expires_at</code>	TIMESTAMPTZ	NULL	Optional expiration date
<code>created_at</code>	TIMESTAMPTZ	DEFAULT <code>now()</code>	Record creation timestamp
<code>updated_at</code>	TIMESTAMPTZ	DEFAULT <code>now()</code>	Last update timestamp

Indexes:

- `invites_access_code_idx` on `access_code` (for fast lookups)
- `invites_email_idx` on `email`
- `invites_status_idx` on `status`

Triggers:

`enforce_invite_amount_bounds()`

- Executes: BEFORE INSERT OR UPDATE
- Enforces: `min >= 5000`, `max <= 50000`, `max >= min`
- Auto-corrects: Sets `min` to 5000 if below, `max` to 50000 if above

Row Level Security:

- Policy: `invites_no_public_access`
 - Effect: All operations blocked for public/anonymous users
 - Access: Service role only (via `supabaseAdmin()`)
-

User Workflows

Workflow 1: Admin Creates Invite

Actor: Admin/CEO

Location: `/hub/foundational/invites`

Steps:

1. Navigate to Hub → Foundational Invites tile
2. Enter recipient email address
3. (Optional) Enter full name
4. (Optional) Enter relationship note
5. (Optional) Set expiration date
6. Click "Create Invite"
7. System generates access code (CRV-XXXX-XXXX-XXXX)
8. Access code displayed in invites table
9. Share access code with recipient via secure channel

Validation:

- Email must be valid format
- Access code is unique (auto-generated)
- Amount limits set automatically (\$50/\$500)

Output:

- Access code: `CRV-XXXX-XXXX-XXXX`
 - Status: `invited`
 - Created timestamp
-

Workflow 2: Recipient Makes Payment

Actor: Invite Recipient

Location: Public pages

Steps:

1. Receive access code from admin
2. Navigate to `https://cravenusa.com/support`
3. Click "Access Portal"
4. Enter access code and email on `/access` page
5. (Optional) Enter relationship note
6. Click "Continue"
7. System verifies code and email

8. Redirected to `/allocate` page
9. Select amount (\$50-\$500) or enter custom amount
10. Accept terms checkbox
11. Click "Continue to Payment"
12. Redirected to Stripe Checkout
13. Complete payment via Stripe
14. Redirected to `/success` page
15. Receive confirmation

Validation Points:

- Access code must match invite
- Email must match invite
- Invite must not be revoked/expired/paid
- Amount must be within bounds
- Terms acceptance required

Error Handling:

- Invalid code: Error message, stay on page
 - Expired/revoked: Error message, cannot proceed
 - Payment failure: Return to `/allocate`, allow retry
-

Workflow 3: Payment Completion (Webhook)

Actor: System (automated)

Location: Stripe webhook → Backend

Steps:

1. Stripe processes payment
2. Stripe sends `checkout.session.completed` event
3. Webhook endpoint receives event
4. Signature verification
5. Extract `invite_id` from metadata
6. Update invite record:
 - Status: `paid`
 - `paid_at`: Current timestamp
 - `paid_amount_cents`: Payment amount
7. Log completion

Error Handling:

- Invalid signature: Reject with 400
 - Missing `invite_id`: Log warning, no update
 - Database error: Log error, Stripe will retry
-

Administrative Procedures

Creating Invites in Bulk

Use Case: Multiple recipients for event or campaign

Procedure:

1. Prepare recipient list (email, name, relationship)
2. For each recipient:
 - Use "Create Invite" form
 - Or use API directly (if automated)
3. Export access codes from invites table
4. Distribute codes via secure channel (email, SMS, etc.)

Best Practices:

- Set expiration dates for time-sensitive campaigns
 - Use relationship notes for tracking
 - Keep access codes confidential
-

Revoking Invites

Use Case: Invite sent in error, recipient no longer eligible

Procedure:

1. Navigate to </hub/foundational/invites>
2. Locate invite in table
3. Click "Revoke" button
4. Confirm action
5. Status updates to "revoked"
6. Recipient cannot use code (returns 403 error)

Limitations:

- Cannot revoke after payment completed
 - Revoked invites remain in database for audit
-

Viewing Payment History

Procedure:

1. Navigate to </hub/foundational/invites>
2. Review invites table
3. Filter by status: "paid"
4. View paid amounts and timestamps
5. Export data if needed (manual copy/paste)

Data Available:

- Access code

- Email
 - Paid amount
 - Payment date (`paid_at`)
 - Created date
-

Troubleshooting Failed Payments

Symptoms:

- Invite shows "accepted" but not "paid"
- Stripe checkout completed but status not updated

Diagnosis:

1. Check Stripe Dashboard for payment status
2. Check webhook logs in Stripe Dashboard
3. Check server logs for webhook errors
4. Verify `STRIPE_WEBHOOK_SECRET` is correct
5. Verify webhook URL is configured in Stripe

Resolution:

1. If webhook failed: Manually update invite status
 2. If payment succeeded: Update via Supabase dashboard
 3. If payment failed: Invite remains "accepted", recipient can retry
-

Security & Compliance

Access Control

Admin Portal:

- Route: `/hub/foundational/invites`
- Guard: `BusinessAuthGuard`
- Permission: Admin or CEO role required
- Torrance: Universal access (bypasses checks)

Public Pages:

- `/support`: No authentication
- `/access`: No authentication (validated via invite)
- `/allocate`: Session-based (from `/access`)
- `/success`: No authentication

Data Protection

Sensitive Data:

- Access codes: Unique, randomly generated
- Email addresses: Stored in database

- Payment amounts: Stored in cents (integers)

Encryption:

- Database: Supabase encryption at rest
- Transmission: HTTPS required (Stripe enforces)
- Session data: Stored in browser sessionStorage (not persistent)

Compliance Requirements

Equity Language Prohibition:

- Public pages (/support) must not contain:
 - "Investment"
 - "Equity"
 - "Shares"
 - "Stock"
 - "Ownership"
 - Any securities-related terminology

Amount Limits:

- Hard-coded: \$50 minimum, \$500 maximum
- Enforced at: Database trigger, API validation, UI constraints
- Cannot be overridden without code changes

Audit Trail:

- All invites tracked with timestamps
- Status changes logged
- Payment amounts recorded
- Revocation actions tracked

Troubleshooting

Issue: Portal Redirects to Home

Symptoms: Clicking foundational invites tile redirects to homepage

Root Cause: Route matching order in React Router

Solution: Ensure /hub/foundational/invites route is defined before /hub route in src/App.tsx

Verification:

```
// Correct order:  
<Route path="/hub/foundational/invites" ... />  
<Route path="/hub" ... />
```

Issue: "Unable to create invite" Error

Symptoms: Create invite button fails with database error

Possible Causes:

1. Database migration not run
2. `SUPABASE_SERVICE_ROLE_KEY` not set
3. RLS policies blocking access

Diagnosis:

1. Check server logs for specific error
2. Verify migration applied: `SELECT * FROM public.invites LIMIT 1;`
3. Verify env var: `echo $SUPABASE_SERVICE_ROLE_KEY`

Resolution:

1. Run migration in Supabase Dashboard
 2. Set `SUPABASE_SERVICE_ROLE_KEY` in `.env`
 3. Restart server
-

Issue: API Calls Return 401 Unauthorized

Symptoms: Frontend cannot load invites list

Root Cause: `assertHubAdmin` function or missing authentication

Current Status: `assertHubAdmin` is a no-op (allows all requests) for MVP

Future Implementation:

- Verify user session
- Check admin/CEO role
- Validate user permissions

Workaround: Ensure user is logged in and has admin access

Issue: Stripe Checkout Not Redirecting

Symptoms: Payment completes but user not redirected to success page

Possible Causes:

1. `NEXT_PUBLIC_APP_URL` or `ORIGIN` not set correctly
2. Stripe session creation failed
3. Network error

Diagnosis:

1. Check browser console for errors

2. Verify Stripe session created: Check server logs
3. Test checkout URL manually

Resolution:

1. Set `ORIGIN=https://cravenusa.com` in `.env`
 2. Verify Stripe keys are correct
 3. Check Stripe Dashboard for session status
-

Issue: Webhook Not Updating Status

Symptoms: Payment succeeds but invite status remains "accepted"

Possible Causes:

1. Webhook not configured in Stripe
2. Webhook secret mismatch
3. Webhook URL incorrect
4. Server not receiving webhooks

Diagnosis:

1. Check Stripe Dashboard → Webhooks → Events
2. Verify webhook URL: <https://cravenusa.com/api/support/webhook>
3. Check server logs for webhook requests
4. Verify `STRIPE_WEBHOOK_SECRET` matches Stripe

Resolution:

1. Configure webhook in Stripe Dashboard
 2. Set correct webhook secret
 3. Test webhook with Stripe CLI or dashboard
 4. Manually update status if needed
-

Maintenance

Regular Tasks

Weekly:

- Review pending invites (status: "invited")
- Check for expired invites
- Verify payment processing

Monthly:

- Export payment reports
- Review revoked invites
- Audit access code distribution

Quarterly:

- Review amount limits (business decision)
- Security audit
- Performance optimization

Database Maintenance

Cleanup:

- Expired invites: Keep for audit (do not delete)
- Revoked invites: Keep for audit (do not delete)
- Paid invites: Archive after 7 years (compliance)

Backup:

- Supabase automatic backups
- Export invites table quarterly
- Store backups securely

Code Maintenance

Dependencies:

- Stripe SDK: Update quarterly
- Express: Security patches immediately
- React: Follow React release cycle

Monitoring:

- API response times
- Error rates
- Payment success rates
- Webhook delivery rates

Appendix

Environment Variables Reference

Variable	Required	Description	Example
SUPABASE_URL	Yes	Supabase project URL	https://xxx.supabase.co
SUPABASE_ANON_KEY	Yes	Supabase anonymous key	eyJ...
SUPABASE_SERVICE_ROLE_KEY	Yes	Service role key (server only)	eyJ...
STRIPE_SECRET_KEY	Yes	Stripe secret key	sk_live_...
STRIPE_WEBHOOK_SECRET	Yes	Webhook signing secret	whsec_...

Variable	Required	Description	Example
ORIGIN	Yes	App base URL	https://cravenusa.com
PORT	No	Server port (default: 3001)	3001

API Response Codes

Code	Meaning	Action
200	Success	Proceed
400	Bad Request	Check request format
401	Unauthorized	Verify authentication
403	Forbidden	Invite invalid/revoked
404	Not Found	Invite doesn't exist
500	Server Error	Check logs, retry

Status Values

Status	Description	Can Proceed to Payment?
invited	Invite created, not yet accessed	No (must verify first)
accepted	Access code verified	Yes
paid	Payment completed	No (already used)
revoked	Invite cancelled by admin	No

Document Control

Approved By: [To be filled]

Review Date: Quarterly

Next Review: May 1, 2025

Change Log:

Date	Version	Changes	Author
2025-02-01	1.0	Initial document creation	System

End of Document